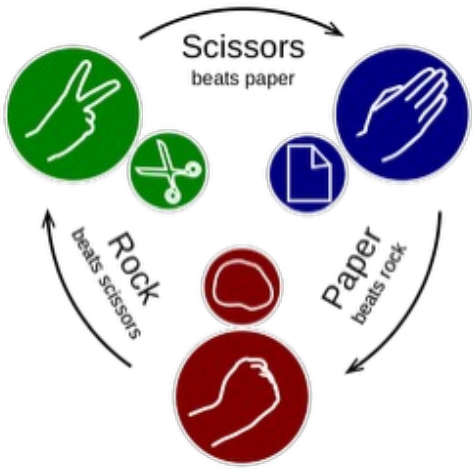
NAME:

Project 2: Rock Paper Scissors (10 pts)

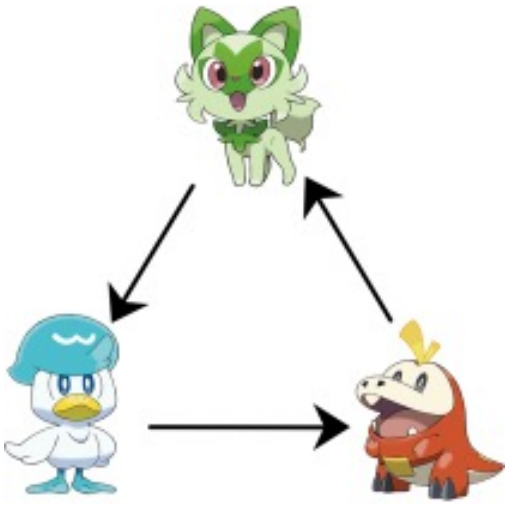
In this project, you'll implement several variants of a text-based Rock-Paper-Scissors game in which a human player plays against the computer. If you're not familiar with the game, check out the Wikipedia article about it: (http://en.wikipedia.org/wiki/Rock-paper-scissors)

Although Rock-Paper-Scissors is a simple game, mechanisms that function similarly to it have been studied in a variety of fields ranging from game theory to biology. One of the most common places to see rock-paper-scissors-like mechanics though is in video games. For example, many interactions between different "types" in the Pokémon games are similar to a game of rock-paper-scissors - the most classic of these being the interactions between the three Pokémon the player can choose at the very beginning of each game in the main series: the options being grass, fire, and water.

The version you'll implement here involves three choices - rock, paper, and scissors. Paper beats (wraps around) rock, scissors beat (cut) paper, and rock beats (blunts) scissors.



(a) RPS



(b) RPS

Learning Objectives:

1. Demonstrate that you can use the programming concepts that we've covered so far. In particular: function calls and definitions, variables and assignment, conditional statements.
2. Practice using random numbers to make random and random, but biased choices.
3. Explore ways to imitate human behavior with the computer.

Reminder: Programming projects should be done individually. You should not look at anyone else's code, show your code to anyone else, write code for anyone else, or let someone else write code for you. Please see the syllabus or Nexus for what to do when you need help with your work.

1. Basic Rock-Paper-Scissors

1.1 Playing one game (2 pts)

1. Open the starter code `rps_1.py` from the course website.
2. Write a function called `rps` that plays a game of rock-paper-scissors with the user. The computer first randomly generates its own move (rock, paper, or scissors) and then asks the user for their choice. Next, it confirms what the user has chosen, reveals its own choice and finally makes one of the following announcements: `"It's a tie", "I win", or "You win"`
3. Make sure that your code is fully commented. Here are a few example interactions:

```
Let's play Rock-Paper-Scissors!
Please type a number to make a choice:
1 -> rock
2 -> paper
3 -> scissors
What do you choose?    1 [Enter] <<< This is the prompt for user input.
You chose: 1 (rock)
I chose: 1 (rock)
It's a tie.
Let's play Rock-Paper-Scissors!
Please type a number to make a choice:
1 -> rock
2 -> paper
3 -> scissors
What do you choose?    1
You chose: 1 (rock)
I chose: 2 (paper)
I win.

Let's play Rock-Paper-Scissors!
Please type a number to make a choice:
1 -> rock
2 -> paper
3 -> scissors
What do you choose?    3
You chose: 3 (scissors)
You chose: 2 (paper)
You win.
```

Hints:

- You probably will want to use the Python `random` module. You can use functions from the module that you've used before (i.e. randrange) or you could use other functions such as randint or choice.
- A link to the documentation for built-in Python modules can be found on the resources page on the course website.
- Feel free to define additional functions if you find it helpful to do so. However, please make sure that `rps` will produce one whole run through the game as shown in the example interactions above.

1.2 Playing until you're tired of it (2 pts)

1. Download the starter file `rps_2.py` from the course website. This file contains a definition for a function called `play`. When play is called with another function name as the parameter value, it will call this function repeatedly until the user indicates that they no longer want to continue.
2. Copy and paste the code for your `rps` function from `rps_1.py` into `rps_2.py`. Follow the instructions in the comments. When you run `rps_2.py`, you should see an interaction similar to the following:

```
Let's play Rock-Paper-Scissors!
Please type a number to make a choice:
1 -> rock
2 -> paper
3 -> scissors
What do you choose?    2
You chose: 2 (paper)
You chose: 3 (scissors)
I win.
Do you want to play again? Type y or n.    y
Great!
Let's play Rock-Paper-Scissors!
Please type a number to make a choice:
1 -> rock
2 -> paper
3 -> scissors
What do you choose?    1
You chose: 1 (rock)
You chose: 3 (scissors)
You win.
Do you want to play again? Type y or n.    n
Ok. See you next time. Bye, bye!
```

2. RPS variations

Choose two (2) of the following four options. For each option, start by making a copy of `rps_2.py`, but save the new code under a new name, as specified by the option you're working on.

2.1 Cheating (3 pts)

Please use the following filename for this variant: `rps_cheating.py`.

In your original RPS program, the computer should choose its move before it asks the human player for their move. But, there's no way for the human player to check whether this is actually what's happening in the program or not. The computer could cheat by waiting for the human to choose a move, then pick a move that beats the human's move.

In this variant, that's what the computer should do. But to not be too obvious, the computer should not cheat every time. Each time the computer has to pick a move, there should be a 1/3 chance that it will cheat. Otherwise, it chooses the move randomly.

2.2 Exploiting human biases (3 pts)

For this variant, download the starter files `rps_anti_human.py` and `game_history.py`. Make sure to save both of these files in the same folder. You'll write all of your code in the file `rps_anti_human.py`.

Studies of humans playing rock-paper-scissors have shown that human players do not always pick their moves randomly. Instead, after they've won, they're more likely to repeat the same move in the next round. In contrast, when they lose, humans are more likely to pick a move to beat the move that they just lost to. For example, if they just played rock and lost to paper, they have a tendency to pick scissors in the next round in order to beat paper.

Knowing about this behavior pattern, you can devise a strategy to beat human players:

1. If I (the computer) just lost (and my human opponent won), I know that the human is likely to play the same move they played in the last round. So I will choose my next move such that it beats their last move.
2. If I (the computer) just won (and my human opponent lost), I know that the human is likely to shift their move to beat my last move. So I will pick my move accordingly. Because of the way rock-paper-scissors works, that means I should now play whatever the human played in the last round.

To implement this strategy, you need a way of remembering the outcome of the last round and what moves were made. The starter file contains a number of functions that you can use for that purpose. These functions refer to code from `game_history.py`. Please do not make any changes to `game_history.py` or to the given function definitions.

References: https://arstechnica.com/science/2014/05/win-at-rock-paper-scissors-by-knowing-thy-opponent/, https://arxiv.org/abs/1404.5199

2.3 Imitating human behavior (3 pts)

For this variant, download the starter files `rps_human_like.py` and `game_history.py`. Make sure to save both of these files in the same folder. You will write all of your code in `rps_human_like.py`.

Studies of humans playing rock-paper-scissors have shown that human players do not always pick their moves randomly. Instead, after they've won, they're more likely to repeat the same move in the next round. In contrast, when they lose, humans are more likely to pick a move to beat the move that they just lost to. For example, if they just played rock and lost to paper, they have a tendency to pick scissors in the next round in order to beat paper.

In this variant, the computer should imitate human behavior. That is:

- If the computer won the last round, it should have a tendency to repeat the same move. However, to not be too predictable, it should only do so with 50% probability. Otherwise it will pick its next move randomly.
- If the computer lost the last round, it should have a tendency to pick a move that defeats the last move by the human player. But again, to not be too predictable, it will only do so with a 50% probability. Otherwise, it will pick its next move randomly.
- If the last game ended in a tie, the computer picks the next move randomly. To implement this strategy, you need a way of remembering the outcome of the last round and what moves were made. The starter file contains a number of functions that you can use for that purpose. These functions refer to code from `game_history.py`. Please do not make any changes to `game_history.py` or to the given function definitions.

2.4 Five options: rock, paper, scissors, Spock, lizard (3 pts)

Start from `rps_2.py` and save it as `rps_five.py`. In this variant you will be implementing a variant of rock-paper-scissors called rock-paper-scissors-Spock-lizard. As the name suggests, it involves 5 options:

- rock beats (blunts) scissors and (crushes) lizard
- paper beats (covers) rock and (disproves) Spock
- scissors beat (cut) paper and (decapitate) lizard
- Spock beats (smashes) scissors and (vaporizes) rock
- lizard beats (poisons) Spock and (eats) paper

Make adjustments to the code originally written for `rps_2.py` so that it implements these five options instead of the original three.

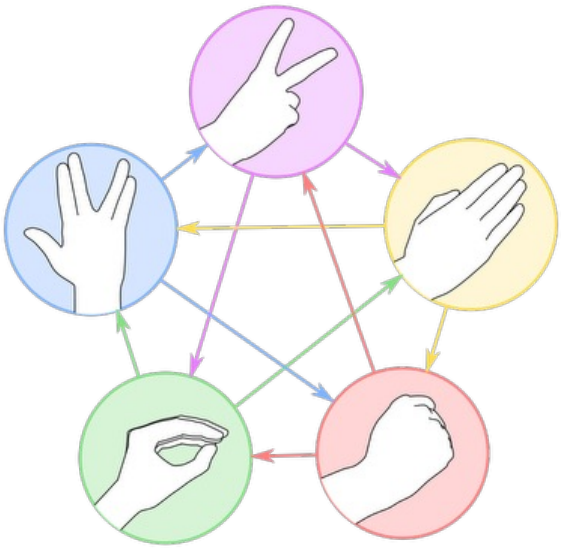


Figure 2: RPS Five

3. What to submit

What to submit You will need to submit the following files:

- `rps_1.py`
- `rps_2.py`

And two of the following (depending on which RPS variants you decide to implement):

- `rps_cheating.py`
- `rps_anti_human.py`
- `rps_human_like.py`
- `rps_five.py`

Before submitting, check that your code meets the following requirements:

1. Are your files properly commented? This should include the following:

a. A header comment, which includes your name and a brief description of the program.
b. Comments within the body of the code to further clarify how the program works.

2. Are your programs formatted neatly and consistently? Do you use some whitespace to help a human reader understand the logical organization of your code? Hint: it may be helpful to think of this as breaking your code up into "paragraphs."
3. Have you cleaned up any code snippets you no longer need?

a. The final version of the program should not contain any lines of actual code that are commented out to keep them from running. Such snippets should be removed before submitting. Once you have checked these things, submit your files to *cat3_Project 2: Rock-Paper-Scissors* on Gradescope.

6. Grading guidelines

- Correctness: programs do what the problems require.
- Program logic: use loops where appropriate, variables where appropriate, and your code doesn't contain lines of code that don't contribute to the program.
- Comments: code is properly commented. There should be a header comment that includes the author's name and describes the program's purpose. Additional comments in the code should help clarify how the program works.
- Organization: use whitespace to indicate the logical structure of the code.
  - Lines of code that logically belong together are close together in the file.
  - Import statements are at the very top…
  - followed by function definitions…
  - then followed by the rest of the code.